Entrega 2: Greedy y Backtracking

Análisis y Diseño de Algoritmos — 2026-2

Caso de Uso: 3.4 Optimización de Tareas en Servidores

Integrantes: Valery Monsalve Correa · Samuel Uribe Salazar

0. CONTEXTO Y CAMBIOS RESPECTO A LA ENTREGA 1

El problema se mantiene igual que en la Entrega 1: un servidor tiene una capacidad de tiempo **T** y recibe **n** tareas, cada una con tiempo de ejecución $\mathbf{t_i}$ y prioridad $\mathbf{p_i}$. Se busca el subconjunto de tareas que maximice la prioridad total sin superar T, con decisión binaria $x_i \in \{0, 1\}$ (formalmente, una mochila 0/1).

0.1 Nuevos conjuntos de datos

En la Entrega 1 los casos mediano y grande eran prefijos del mismo listado de tareas, con prioridades poco relacionadas con el tiempo. Con esos datos el greedy coincidía con el óptimo en el caso grande y el backtracking terminaba en 21 nodos, así que la comparación entre técnicas no mostraba diferencias. Por eso se creó **src/generar_datos.py**, que genera casos reproducibles (semilla fija) con estas características:

- La prioridad está **correlacionada con el tiempo** ($p_i = t_i$ + ruido entre −3 y 12): una tarea larga suele ser más importante. Es más realista y es justamente el tipo de instancia donde el greedy se equivoca.
- La capacidad es el **45 % de la suma de tiempos**, lo que deja muchísimas combinaciones válidas (peor escenario para la fuerza bruta).
- Las semillas se escogieron para que el greedy **no** alcance el óptimo, de modo que la diferencia de calidad sea visible.

Archivo	n	T	Cambio
caso_pequeno.txt	10	50	Sin cambios (verificado a mano en la Entrega 1)
caso_mediano.txt	20	159	Regenerado (semilla 202)
caso_grande.txt	28	241	Regenerado (semilla 391). En el mismo equipo la fuerza bruta pasa de 17.7 s (caso anterior) a 49.5 s y recorre 203.8 millones de nodos
caso_contraejemplo_greedy.txt	3	10	Nuevo: contraejemplo mínimo del greedy

Todas las mediciones de este documento se tomaron ejecutando src/comparativa.py en el mismo equipo. Los tiempos absolutos pueden variar entre computadores, pero las proporciones entre técnicas se mantienen.

1. SOLUCIÓN GREEDY

1.1 Decisión local óptima

La pregunta que el greedy responde en cada paso es: *¿qué tarea entrega más prioridad por cada unidad de tiempo de servidor que consume?* Por eso el criterio es la **densidad** $r_i = p_i / t_i$. Se ordenan las tareas de mayor a menor densidad y se recorren una sola vez: si la tarea cabe en la capacidad restante se toma, si no, se descarta. Una vez tomada una decisión no se reconsidera.

Se descartaron otros criterios locales: **mayor prioridad primero** tiende a llenar el servidor con pocas tareas largas, y **menor tiempo primero** ignora la prioridad. La densidad combina ambos factores y es el criterio que resuelve de forma óptima la versión fraccional del problema, por lo que es la elección natural.

1.2 Implementación (src/greedy.py)

```
def greedy(tareas, capacidad):
    ordenadas = sorted(tareas, key=lambda t: t.prioridad / t.tiempo, reverse=True)  # O(n log n)
    prioridad_total, tiempo_usado, seleccionadas = 0, 0, []
    for tarea in ordenadas:                                    # O(n)
        if tiempo_usado + tarea.tiempo <= capacidad:
            seleccionadas.append(tarea.id)
            tiempo_usado += tarea.tiempo
            prioridad_total += tarea.prioridad
    return prioridad_total, sorted(seleccionadas), tiempo_usado
```

1.3 Análisis de complejidad

- **Ordenamiento:** sorted() de Python (Timsort) cuesta O(n log n).
- **Recorrido:** un ciclo de n iteraciones con operaciones O(1) cada una: O(n).
- **Total temporal:** T(n) = O(n log n) + O(n) = **O(n log n)**.
- **Espacial:** la lista ordenada y la lista de seleccionadas ocupan como máximo n elementos: **O(n)**.

Empíricamente el greedy resuelve n = 1000 en ~0.35 ms, mientras que la fuerza bruta no termina para n = 50 ($2^{50} \approx 10^{15}$ combinaciones).

1.4 Análisis crítico: ¿es óptimo?

No. El greedy por densidad es óptimo solo para la versión *fraccional* (cuando se puede ejecutar parte de una tarea). En nuestro problema las tareas son indivisibles, y una decisión localmente buena puede dejar tiempo de servidor desperdiciado que una combinación distinta sí habría aprovechado.

Contraejemplo mínimo (datos/caso_contraejemplo_greedy.txt), con T = 10:

Tarea	Tiempo	Prioridad	Densidad
1	6	7	1.17
2	5	5	1.00
3	5	5	1.00

- **Greedy:** toma la tarea 1 (mayor densidad, 6 de tiempo). Quedan 4 unidades y ni la 2 ni la 3 caben. Resultado: **prioridad 7**, con 4 unidades sin usar.
- **Óptimo:** tareas 2 + 3, que usan exactamente las 10 unidades. Resultado: **prioridad 10**. El greedy obtiene solo el 70 % del óptimo.

El mismo fenómeno ocurre en nuestros casos de prueba:

Caso	Greedy	Óptimo	% del óptimo	Por qué falla
Pequeño (T=50)	85 (usa 49/50)	90	94.4 %	Toma 6, 9, 2 y 7 (densidades altas); luego la tarea 1 ya no cabe y rellena con la 5 (densidad 1.0). El óptimo {1, 6, 9, 10} llena exactamente las 50 unidades.
Mediano (T=159)	238	255	93.3 %	Elige tareas densas pero pequeñas y deja huecos donde no entra ninguna tarea restante.
Grande (T=241)	362 (usa 226/241)	378	95.8 %	Incluye la tarea 7 (12 de tiempo, prioridad 16) y deja 15 unidades libres. El óptimo cambia la 7 por la 21 (27 de tiempo, prioridad 32): ocupa las 241 unidades y gana 16 puntos más.

¿En qué casos falla? Cuando la capacidad sobrante al final no alcanza para ninguna tarea restante, es decir, cuando la suma de tiempos de las tareas densas deja un "hueco" que otra combinación sí habría llenado. Por eso falla más cuando las tareas son grandes respecto a T (cada hueco pesa más). **¿Cuándo acierta?** Si todas las tareas tienen el mismo tiempo, si las tareas elegidas llenan exactamente la capacidad, o si se permitiera ejecutar fracciones de tareas.

Sin embargo, el error es acotado en la práctica: en las instancias aleatorias el greedy obtuvo entre el 88.6 % y el 100 % del óptimo, y con n ≥ 200 siempre superó el 99.8 %, porque el hueco final pesa muy poco frente a una prioridad total grande (ver gráfica de calidad en la sección 3).

2. SOLUCIÓN CON BACKTRACKING

2.1 Estrategia

El backtracking recorre el mismo árbol binario de decisiones que la fuerza bruta (para cada tarea: *incluir* o *excluir*), pero construye la solución paso a paso y **abandona una rama en cuanto sabe que no puede llevar a una solución válida o mejor** que la mejor encontrada hasta el momento. Al volver de una rama deshace la decisión (actual.pop()), que es el "backtrack".

2.2 Restricciones que permiten podar

- **Poda 1 — Factibilidad (capacidad):** la rama *incluir* solo se abre si $t_i \leq$ capacidad restante. Elimina todos los subconjuntos que superan T.
- **Poda 2 — Cota superior (optimalidad):** en cada nodo se calcula una estimación optimista de lo máximo que aún se podría ganar: se llenan las horas restantes con las tareas que faltan en orden de densidad, permitiendo tomar una *fracción* de la última. Si **prioridad_actual + cota ≤ mejor_encontrada**, nada en ese subárbol puede mejorar el resultado y se descarta completo.
- **Ordenamiento previo por densidad:** las tareas se exploran de mayor a menor densidad, lo que hace la cota más ajustada y encuentra soluciones buenas muy pronto.
- **Solución inicial = greedy:** la mejor solución arranca con el resultado del greedy, de modo que la Poda 2 corta desde la raíz.

La Poda 2 nunca descarta el óptimo: la cota fraccional siempre es mayor o igual que lo que cualquier selección entera puede lograr, así que si ni siquiera la cota supera a la mejor solución, ninguna solución real de esa rama lo hará. Por eso el backtracking **garantiza la solución**

óptima, igual que la fuerza bruta (comparativa.py lo verifica con un assert en cada instancia).

2.3 Implementación (src/backtracking.py, núcleo)

```
def explorar(indice, cap_restante, prioridad_actual):
    nodos[0] += 1
    if prioridad_actual > mejor["prioridad"]:              # nueva mejor solución
        mejor["prioridad"], mejor["ids"] = prioridad_actual, list(actual)
    if indice == n or cap_restante == 0:
        return
    if usar_cota and prioridad_actual + cota_superior(indice, cap_restante) <= mejor["prioridad"]:
        return                                             # PODA 2: cota
    tarea = orden[indice]
    if tarea.tiempo <= cap_restante:                       # PODA 1: capacidad
        actual.append(tarea.id)
        explorar(indice + 1, cap_restante - tarea.tiempo, prioridad_actual + tarea.prioridad)
        actual.pop()                                       # backtrack
    explorar(indice + 1, cap_restante, prioridad_actual)   # rama excluir
```

2.4 Análisis de complejidad

Peor caso. Si ninguna poda actúa, el árbol es el mismo de la fuerza bruta: $2^{n+1} - 1$ nodos. Además, en cada nodo se calcula la cota, que recorre hasta n tareas: $T(n) = 2 \cdot T(n-1) + O(n)$, cuya solución es **$O(n \cdot 2^n)$**. Sumando el ordenamiento inicial O(n log n), el peor caso sigue siendo exponencial. Un ejemplo de peor caso son instancias donde muchas combinaciones tienen casi la misma prioridad, de modo que la cota casi nunca logra descartar.

Caso promedio (con poda). No existe una fórmula cerrada porque depende de los datos, así que se midió contando los nodos visitados:

Caso	n	Nodos fuerza bruta	Nodos BT	% del árbol visitado	Tiempo FB	Tiempo BT
Contraejemplo	3	11	9	—	0.00002 s	0.00003 s
Pequeño	10	613	48	7.8 %	0.0003 s	0.0001 s
Mediano	20	830,914	74	0.009 %	0.326 s	0.0006 s
Grande	28	203,848,973	146	0.00007 %	49.50 s	0.00016 s

En el caso grande el backtracking visita 146 nodos en lugar de 203.8 millones: es **unas 300,000 veces más rápido** que la fuerza bruta y entrega exactamente la misma prioridad (378).

¿Cuánto aporta cada poda? En la curva de la sección 3 se ejecutó también una versión con *solo* la poda por capacidad (usar_cota=False). Para n = 25 visitó 25.3 millones de nodos y tardó 7.5 s, prácticamente lo mismo que la fuerza bruta (6.5 s), mientras que con la cota visitó 160 nodos (0.0002 s). Conclusión: con T = 45 % de la suma de tiempos casi todas las combinaciones caben, por lo que la poda por capacidad corta muy poco; **la ganancia real viene de la poda por cota**.

Límite observado. Con n = 1000 el backtracking visitó 19.4 millones de nodos y tardó ~14 s, frente a 0.35 ms del greedy. Esto confirma experimentalmente que, aunque la poda es muy eficaz en promedio, el algoritmo sigue siendo exponencial en el peor caso.

Espacial: la recursión tiene profundidad máxima n y en cada nivel solo se guardan variables O(1); la solución actual y la mejor ocupan O(n). Total **O(n)**.

3. COMPARATIVA PARCIAL

3.1 Tabla comparativa de las tres soluciones

Criterio	Fuerza bruta	Greedy	Backtracking
Complejidad temporal	$O(2^n)$	O(n log n)	$O(n \cdot 2^n)$ peor caso; muy inferior en promedio
Complejidad espacial	O(n)	O(n)	O(n)
¿Solución óptima?	Sí, siempre	No (88.6 %–100 % del óptimo en las pruebas)	Sí, siempre
Tiempo caso grande (n=28)	49.50 s	0.00003 s	0.00016 s
Prioridad caso grande	378	362	378
Mayor n práctico	≈ 28–30	Millones	Cientos (n=500 en 0.05 s; n=1000 ya tarda ~14 s)
Facilidad de implementación	Alta: recursión directa	Muy alta: ordenar + un ciclo	Media: requiere diseñar y justificar la cota

3.2 Tiempos en los archivos de prueba

Archivo	n	T	FB prioridad	FB tiempo	Greedy prioridad	Greedy tiempo	BT prioridad	BT tiempo
Contraejemplo	3	10	10	0.00002 s	7 ✗	0.000006 s	10	0.000025 s
Pequeño	10	50	90	0.0003 s	85 ✗	0.000007 s	90	0.000104 s
Mediano	20	159	255	0.3260 s	238 ✗	0.000028 s	255	0.000556 s
Grande	28	241	378	49.4985 s	362 ✗	0.000031 s	378	0.000164 s

✗ = el greedy no alcanzó el óptimo.

3.3 Gráficas

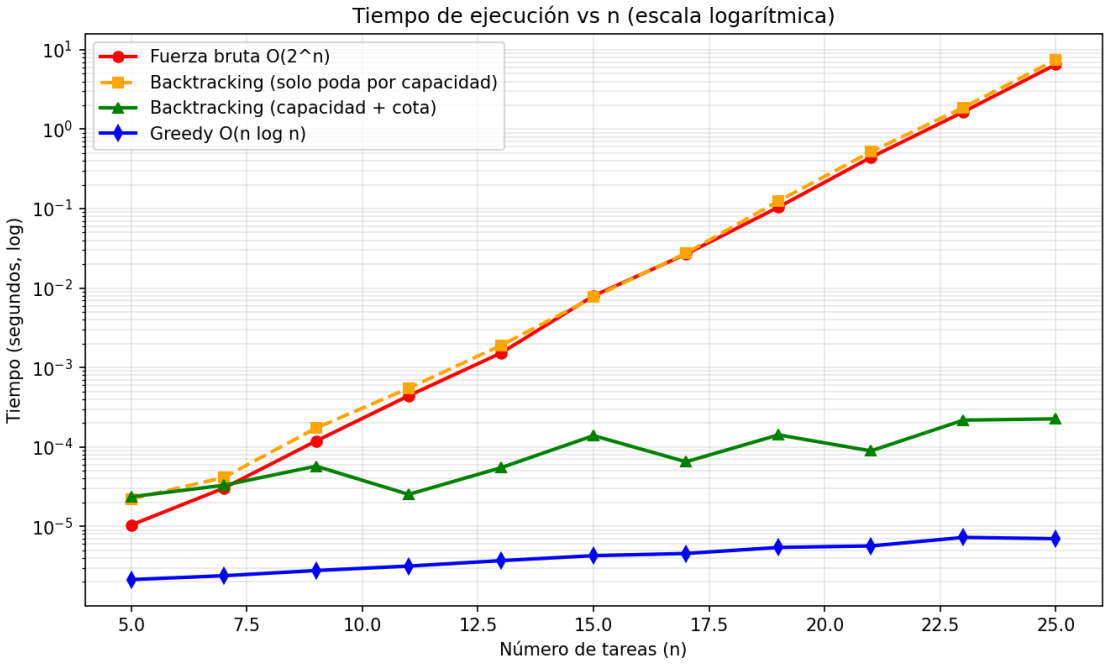


Figura 1. Tiempo vs n (escala logarítmica) con instancias aleatorias n = 5..25. La fuerza bruta y el backtracking con solo poda por capacidad crecen como rectas en escala log (exponencial); el backtracking con cota y el greedy se mantienen planos.

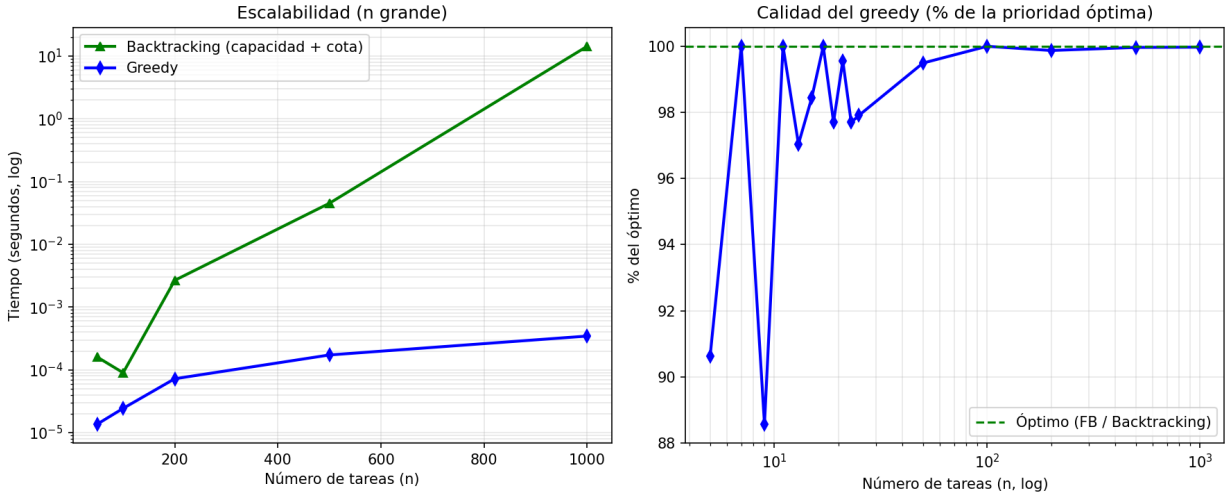


Figura 2. Izquierda: greedy vs backtracking para n = 50..1000 (la fuerza bruta es inviable). Derecha: porcentaje del óptimo que alcanza el greedy según n.

3.4 Discusión de trade-offs

- **¿Cuál es más rápida?** El greedy, por varios órdenes de magnitud y en todos los tamaños. El backtracking es casi igual de rápido hasta unos cientos de tareas, pero su tiempo es impredecible: de n = 500 (0.05 s) a n = 1000 (14 s) se multiplicó por casi 300.
- **¿Cuál da mejor resultado?** Fuerza bruta y backtracking, que siempre encuentran el óptimo. El greedy pierde entre 0 % y 11 % de prioridad, y la pérdida es mayor cuando hay pocas tareas grandes respecto a la capacidad.

- **¿Cuál es más fácil de implementar?** El greedy (ordenar y un ciclo). La fuerza bruta también es simple. El backtracking es el más delicado: una cota mal planteada puede descartar el óptimo sin que el programa falle, por eso se validó contra la fuerza bruta.
- **Recomendación actual:** para pocas decenas o cientos de tareas, backtracking (óptimo y prácticamente instantáneo). Para miles de tareas o cuando hay que decidir en tiempo real, greedy, aceptando una pérdida pequeña. La fuerza bruta solo sirve como referencia para validar a las demás. En la Entrega 3 se evaluará si divide y vencerás y programación dinámica (O(n·T)) ofrecen un óptimo garantizado con tiempo predecible.

4. EJECUCIÓN

```
python src/greedy.py          # greedy sobre los 4 archivos de datos
python src/backtracking.py     # backtracking sobre los 4 archivos de datos
python src/comparativa.py      # tabla + gráficas (≈1 minuto, incluye fuerza bruta n=28)
python src/comparativa.py --rapido   # omite la fuerza bruta del caso grande (para la demo)
python src/generar_datos.py    # regenera caso_mediano.txt y caso_grande.txt
```